Agentic AI-Assisted Autonomous Orchestration for 5G Network Slicing: A LangGraph Multi-Agent Framework with Chain-of-Thought Reasoning and Wrong-Lever Avoidance

Simran R. Kalkeri[1], Amaanali D. Doddamani[1], Sparsh S. Naik[1], Rishi P. Kulkarni[1], and Narayan D. G.*[1]

[1]*School of Computer Science and Engineering, KLE Technological University, Hubballi – 580031, Karnataka, India.*

**Corresponding author. E-mail: narayan.dg@kletech.ac.in;*

Contributing authors: simran.kalkeri@kletech.ac.in; amaanali.doddamani@kletech.ac.in; sparsh.naik@kletech.ac.in; rishi.kulkarni@kletech.ac.in;

Abstract

Fifth-generation (5G) network slicing enables the concurrent operation of enhanced Mobile Broadband (eMBB), Ultra-Reliable Low-Latency Communication (URLLC), and massive Machine-Type Communication (mMTC) services on shared physical infrastructure. Static threshold-based controllers, while operationally simple, fail under dynamic traffic conditions: they apply control actions without causal reasoning, waste bandwidth by throttling idle slices, and retain no operational memory across control cycles. This paper presents an autonomous, cloud-native agentic orchestration framework for 5G network slice management. The system integrates a LangGraph-based multi-agent pipeline with a locally hosted Large Language Model (LLM)—specifically `llama3.2:3b` via Ollama—to perform structured Chain-of-Thought (CoT) reasoning for slice QoS enforcement. The framework contributes a five-VM cloud-native 5G testbed combining Open5GS, UERANSIM, and Kubernetes-hosted per-slice User Plane Functions (UPFs); a decoupled multi-agent architecture spanning Monitoring, State, Planning, Validation, and Execution agents; a mandatory JSON CoT schema requiring root-cause assessment and lever-validity scoring before any control action; a Wrong-Lever Avoidance (WLA) mechanism using the novel ρ_{eMBB} load-fraction metric to suppress spurious throttling; and an episodic memory subsystem enabling memory-assisted adaptive decision-making. Experimental evaluation against a rule-based baseline demonstrates that the agentic orchestrator improves URLLC SLA compliance from 87.1% to 96.3%, halves mean recovery time from 8.4 s to 4.2 s, reduces unnecessary eMBB throughput loss by 72%, and maintains decision overhead below 380 ms—well within the 2-second macroscopic control window.

Keywords: 5G network slicing, agentic AI, LLM orchestration, LangGraph, QoS enforcement, URLLC, eMBB, chain-of-thought reasoning, cloud-native, Kubernetes, Open5GS, multi-agent systems

1 Introduction

Fifth Generation (5G) mobile networks introduce network slicing as a defining architectural capability, enabling multiple logical network instances—each with distinct performance characteristics—to coexist on shared physical infrastructure [?]. Network slicing permits concurrent support of enhanced Mobile Broadband (eMBB), Ultra-Reliable Low-Latency Communication (URLLC), and massive Machine-Type Communication (mMTC) services, each with fundamentally different Quality-of-Service (QoS) requirements. eMBB demands high throughput and tolerates moderate latency; URLLC requires end-to-end latencies below 20 ms with very high reliability; and mMTC targets energy-efficient transmission for large-scale IoT device populations.

Realizing these guarantees in practice is challenging. At the user plane, traffic from different slices may contend for shared buffering and forwarding resources during congestion. Static resource allocations fail to track dynamic traffic demands, while reactive threshold-based controllers lack the ability to reason about the root cause of degradation or to anticipate violations before they occur [?]. Autonomous orchestration, enabled by Large Language Models (LLMs) and multi-agent frameworks, offers a qualitatively different control paradigm. Rather than pattern-matching against hardcoded thresholds, an LLM-based planning agent performs causal reasoning: identifying which network entity is responsible for congestion, validating whether the proposed control action addresses that cause, and consulting prior operational history stored in an agent memory to refine its decisions [?, ?]. This paper implements, deploys, and evaluates such a system in a cloud-native 5G testbed.

Motivation. The evolution from rule-based to autonomous orchestration is driven by three fundamental limitations observed in practical 5G slice management systems. **First**, rule-based controllers are structurally blind to causation: a threshold-based system that detects elevated URLLC Round-Trip Time (RTT) will throttle eMBB bandwidth regardless of whether eMBB traffic is actually active. When eMBB load is low, this action is unnecessary and harmful. An agent capable of reasoning about relative load—quantified by the ρ_{eMBB} metric introduced in this work—can correctly distinguish idle from bursting eMBB and decline to act when throttling is ineffective. **Second**, reactive control cannot prevent violations; it can only respond to them. A URLLC SLA breach lasting even 200 ms may disrupt a latency-critical industrial automation session, making pre-emptive, trend-aware intervention essential [?]. **Third**, accumulated operational experience is wasted in memoryless systems: an agent that stores the outcome of prior actions can converge to more accurate control policies over time without requiring offline training. These three limitations collectively motivate the agentic, causally-grounded, memory-augmented design proposed in this work.

Contributions. This paper makes five interconnected contributions to autonomous 5G slice orchestration. First (C1), we build a five-VM cloud-native 5G research testbed integrating Open5GS, UERANSIM, Kubernetes-hosted per-slice UPFs, and Prometheus/ Grafana monitoring, providing the live experimental substrate on which all claims are validated. Second (C2), we design a LangGraph-based multi-agent orchestration pipeline with a monitoring thread that is fully decoupled from LLM inference, ensuring continuous 3-second metric freshness regardless of planning latency. Third (C3), we introduce a mandatory Chain-of-Thought JSON reasoning schema that forces the LLM to produce a root-cause attribution and lever-validity score before selecting any control action, making every decision fully auditable. Fourth (C4), we propose a Wrong-Lever Avoidance

(WLA) mechanism anchored on the dimensionless ρ_{eMBB} load-fraction metric, which deterministically suppresses spurious throttling of idle slices and is the primary driver of throughput preservation gains. Fifth (C5), we contribute an episodic memory subsystem that encodes recent action outcomes into the LLM planning context, enabling the agent to adapt its strategy based on live operational history without any offline training.

The remainder of this paper is organized as follows. Section **??** reviews related work and identifies the research gap. Section **??** describes the testbed, the seven-layer architecture, and the full agentic framework. Section **??** presents and analyses experimental results across all five contributions. Section **??** concludes and outlines future directions.

2 Background Work

Research in autonomous 5G slice management spans three broadly overlapping themes: dynamic orchestration of core-network functions, AI and reinforcement-learning driven resource allocation, and LLM-assisted intent-based management. We review representative work in each category before identifying the gap that motivates our design.

2.1 Dynamic Network Slice Orchestration

Grings et al. [**?**] propose a full dynamic orchestration framework for 5G core network slicing over a cloud-native Kubernetes platform. Unlike prior approaches that support only partial orchestration through resource scaling, their solution introduces a Kubernetes-integrated controller capable of dynamically reconfiguring 5G core functions—including NSSF, AMF, SMF, and UPF—without terminating active slices. Evaluation on a free5GC-based testbed showed a 47.5% reduction in slice reconfiguration requests compared to partial orchestration. Novanana et al. [**?**] investigate Kubernetes-based MANO for provisioning coexisting eMBB and URLLC services using free5GC, UERANSIM, and OpenStack. Comparing single-UPF and dedicated SMF+UPF scenarios, they demonstrate that dedicated isolation delivers more stable throughput (20.2–22.1 Gbps) and fewer retransmissions—a finding that directly motivates our per-slice UPF deployment strategy.

2.2 AI and Reinforcement Learning for Slice Management

Reddy [**?**] proposes a Deep Reinforcement Learning (DRL)-based Kubernetes autoscaling framework achieving 71.7% faster slice deployment, 35% higher resource utilization, and 22% lower latency versus threshold-based autoscaling. Sulaiman et al. [**?**] present MicroOpt, combining DNN-based traffic prediction with gradient optimization to achieve up to 21.9% lower resource consumption than state-of-the-art methods. Saha et al. [**?**] design MonArch, a scalable monitoring architecture for cloud-native 5G deployments, demonstrating that a 5-second Prometheus monitoring interval provides an effective accuracy-overhead balance. While these approaches yield strong results in simulation or partial deployments, they require offline training data and cannot express the causal reasoning necessary to avoid wrong-lever interventions.

2.3 LLM-Assisted Network Management

Park et al. [**?**] demonstrate LLM-based network management spanning intent specification through to configuration generation. Dandoush et al. [**?**] propose a conceptual

LLM-powered framework integrating multi-agent systems with ETSI MANO and 3GPP slicing architectures for natural-language intent translation, though without experimental validation. Bandara et al. [?] present an agentic AI control plane for 6G slice orchestration using the Model Context Protocol (MCP), demonstrating successful LLM fine-tuning and reliable tool invocation on an Open5GS, Ericsson RAN, and Kubernetes testbed. Tran et al. [?] propose PCLANSA, an LSTM-based closed-loop service assurance framework for 5G/B5G slicing achieving resource savings of 54.85% for eMBB and 57.1% for URLLC versus static provisioning.

2.4 Research Gap

Existing approaches either rely on deterministic rule engines that lack causal reasoning, offline-trained DRL agents that require extensive simulation data, or conceptual LLM frameworks that have not been validated on live networks. None provides an LLM-based, causally-grounded, memory-augmented orchestration system evaluated on a live cloud-native 5G testbed with real user-plane isolation simultaneously across three slice types. Our work fills this gap by combining live testbed operation, structured causal reasoning, deterministic safety gating, and episodic memory into a single coherent framework.

3 System Architecture

This section describes the complete system from physical infrastructure through to the agentic decision loop. The design philosophy is layered: the lower layers provide the 5G substrate and traffic isolation machinery, while the upper layers implement the autonomous control intelligence. Figure **??** shows the overall architecture of the proposed system.

[Image Placeholder: system_archie_3.png]

Figure 1: End-to-end architecture of the proposed Agentic 5G Slice Orchestrator. User traffic generated by UERANSIM traverses the Open5GS 5G core, per-slice UPFs, and Linux `tc` shaping before reaching MEC applications. The agentic orchestration layer on kubemaster closes the control loop via Prometheus, SSH, and the Kubernetes API.

3.1 Testbed Infrastructure

The testbed is deployed on a single physical host running VMware, hosting five virtual machines sharing a flat Layer-2 network (192.168.49.0/24). Table **??** summarizes the configuration of each VM. The physical host provides Intel Xeon Gold 5418Y processors, and the worker node (kube) is deliberately over-provisioned with 16 vCPUs and 62 GiB RAM to accommodate simultaneous UPF pods, MEC application pods, and heavy traffic load without resource contention.

Table 1: Virtual Machine Configuration of the 5G Testbed

VM / Role	IP Address	vCPU / RAM	Key Components
kubemaster (K8s CP)	192.168.49.174	8c / 15 GiB	Prometheus, Grafana, Orchestrator, Ollama
kube (K8s Worker 1)	192.168.49.173	16c / 62 GiB	UPF pods (hostNet), App pods, metrics-exporter
kube2 (K8s Worker 2)	192.168.49.181	8c / 15 GiB	Secondary worker (standby)
shinegami (5G Core)	192.168.49.143	8c / 62 GiB	Open5GS: AMF, 3×SMF, NRF, NSSF, UDM
UERANSIM (RAN/UE)	192.168.49.139	8c / 62 GiB	nr-gnb, 3×nr-ue, 9×uesimtun

Three UE processes are instantiated—one per slice type (eMBB, URLLC, mMTC)—and each establishes three PDU sessions, creating nine virtual tunnel interfaces (`uesimtun0`–`uesimtun8`). Traffic isolation is enforced by binding client sockets to slice-specific tunnel interfaces, ensuring that slice traffic never crosses a UPF boundary.

3.2 Seven-Layer Architectural Model

The end-to-end system is organized into seven logical layers aligned with 3GPP and cloud-native MEC paradigms. Working from the bottom up, the first three layers constitute the standard 5G protocol stack, the next two provide cloud-native hosting and QoS enforcement, and the top two implement the intelligence and visibility planes.

L1. User Plane / Traffic Clients — Three Python clients generate HLS video streaming (`embb_client.py`, ≈43.7 Mbps), RTT probes (`urllc_client.py`, baseline 12–16 ms), and MQTT IoT sensor data (`mmtc_client.py`), each bound to the appropriate `uesimtun` interfaces.

L2. Radio Access Network — UERANSIM `nr-gnb` implements a software-defined 5G gNodeB for PLMN 999:70. Three UE processes perform 3GPP 5G-AKA authentication and establish nine PDU sessions spanning S-NSSAIs 1, 2, and 3.

L3. 5G Core Network — Open5GS provides a full 3GPP SA control plane: a single AMF, three dedicated SMFs (one per slice), and supporting NRF, NSSF, UDM, UDR, AUSF, and PCF functions communicating via HTTP/2 Service-Based Interfaces (SBI).

L4. Kubernetes Data Plane & MEC — Three UPF pods deployed with `hostNetwork: true` terminate N3 GTP-U tunnels on dedicated host IPs, creating per-slice `ogstun` tunnel devices. Co-located application pods (nginx, Node-RED, Mosquitto) serve as MEC applications.

L5. QoS Enforcement — Linux Traffic Control (`tc`) with Hierarchical Token Bucket (HTB) queuing enforces per-slice bandwidth policies on `ogstun-embb`, dynamically modifiable by the orchestrator via SSH without pod restarts.

L6. Observability Stack — A custom `tun-metrics-exporter` DaemonSet exposes `/proc/net/dev` counters to Prometheus (NodePort 30090), visualized through Grafana (NodePort 30300) at 5-second scrape intervals.

L7. Agentic Orchestration — The LangGraph multi-agent system on kubemaster implements the MAPE-K control loop, querying Prometheus via PromQL, reasoning with a locally hosted LLM, and enforcing decisions via SSH and the Kubernetes API.

3.3 Slice Configuration

Each slice is uniquely identified by an S-NSSAI tag and receives its own dedicated SMF and UPF instances, preventing any signaling or data-plane cross-contamination. Table **??** details the mapping from slice type to network resources and MEC applications.

Table 2: 5G Network Slice Configuration Parameters

Slice	S-NSSAI	UPF IP Pool	SMF Service	Application
eMBB	SST=1, SD=0x000001	10.45.0.0/16	smfd-embb	nginx (:30880)
URLLC	SST=2, SD=0x000002	10.46.0.0/16	smfd-urllc	Node-RED (:30180)
mMTC	SST=3, SD=0x000003	10.47.0.0/16	smfd-mmtc	Mosquitto (:30883)

3.4 Agentic Orchestration Framework

The orchestration intelligence is implemented as a stateful LangGraph graph on kubemaster. Unlike the static configuration layers below, this layer is inherently dynamic: it continuously monitors the network, reasons about observed conditions, and issues targeted control actions to restore or maintain SLA compliance. The framework comprises five agents organized as a directed graph, with an optional memory feedback edge that converts it into a closed loop. The orchestration decision lifecycle is illustrated in Fig. **??**.

[Image Placeholder: decision_lifecycle.png]

Figure 2: Orchestration decision lifecycle: an eMBB surge triggers cross-slice contention and a URLLC SLA breach; the agentic framework autonomously detects the root cause, issues a throttle command, restores RTT below threshold, and lifts the rate cap once stable—forming a continuous closed-loop control cycle.

Monitoring Agent (C2). A Python daemon thread polling Prometheus PromQL endpoints every $\Delta = 3\,\text{s}$, independent of LLM inference latency. Simultaneously, it parses RTT telemetry logs from the UERANSIM VM via SSH and reads the current `tc` enforcement state. The agent writes atomically to a thread-safe state cache $\mathcal{S}$, ensuring the planning agent always reads a consistent snapshot.

State Agent. Receives $\mathcal{S}$ and enriches it with derived signals: consecutive SLA violation count, RTT trend direction (rising/falling), and current enforcement status (`NORMAL`/`THROTTLED`). These derived signals are critical inputs to both the memory retrieval step and the LLM prompt.

Planning Agent (C3). The cognitive core of the framework. It constructs a structured prompt from the current state vector and the last $K = 10$ memory episodes, then invokes `llama3.2:3b` via the Ollama REST API. The mandatory JSON CoT schema enforces five output fields: (i) `root_cause_assessment` — a causal analysis of the observed SLA breach; (ii) `lever_validity` — justification for whether throttling eMBB addresses the root cause; (iii) `confidence` in [0.0, 1.0]; (iv) `action` drawn from {`throttle_embb`, `restore_embb`, `no_action`}; and (v) `new_rate_mbit` in [50, 1000]. If the LLM returns malformed output or times out after 60 s, a deterministic rule-based fallback activates transparently.

Validation Gate / Wrong-Lever Avoidance (C4). Implements the ρ_{eMBB} load-fraction check (Eq. **??**) and scans `root_cause_assessment` for contradiction keywords $\mathcal{K}_{\text{deny}} = \{$"not embb", "low load", "nearly idle", "not the cause"$\}$. A contradictory throttle action has its confidence capped at 0.35. The gate also clamps the proposed rate to $[r_{\min}, r_{\max}] = [50, 1000]\,\text{Mbit/s}$ and blocks redundant actions when the proposed rate equals the currently enforced rate.

Execution Agent. Issues the validated `tc` command over SSH to the worker node. A throttle action applies a Token Bucket Filter queue; a restore action deletes it. For MEC application scaling, the agent uses the Kubernetes API (`kubectl scale deployment`).

Memory Agent (C5). After each executed action, the agent records the outcome delta $\Delta R = R_{\text{post}} - R_{\text{pre}}$ and classifies it as `improved` ($\Delta R < -2\,\text{ms}$), `worsened` ($\Delta R > 2\,\text{ms}$), or `neutral`. The sliding-window episodic store of depth $K = 10$ is serialized into the LLM planning prompt at the next cycle, enabling the agent to adapt its strategy based on observed operational history.

3.5 Mathematical Foundations

The agentic framework is grounded in four performance metrics that together operationalize its control objectives. Each metric is defined formally below; all symbols and their units are listed immediately after each equation to ensure self-contained readability.

3.5.1 eMBB Load Fraction.

The eMBB relative load fraction ρ_{eMBB} is the central enabler of the Wrong-Lever Avoidance mechanism (C4). Unlike an absolute throughput threshold, this dimensionless ratio is invariant to baseline traffic volume, allowing the orchestrator to reliably identify genuine congestion regardless of application-layer bit-rate [**?**]. It computes the current packet rate as a fraction of the recent historical peak:

$$\rho_{\text{eMBB}}(t) = \frac{\lambda(t)}{\max\limits_{t' \in [\,t - W\Delta,\; t\,]} \lambda(t')} \tag{1}$$

Here, $\rho_{\text{eMBB}}(t) \in [0, 1]$ is the dimensionless eMBB relative load fraction at time t; $\lambda(t)$ is the eMBB packet rate (pkt/s) observed at the current cycle; $\lambda(t')$ is the packet rate at any past sample t' within the rolling window; $W = 8$ is the observation window size in monitoring cycles; $\Delta = 3\,\text{s}$ is the monitoring interval between successive samples; and $t - W\Delta$ marks the start boundary of the rolling observation window.
A value $\rho_{\text{eMBB}}(t) < 0.2$ classifies the eMBB slice as idle at time t, causing the Validation Gate to suppress any throttling action regardless of the observed URLLC RTT. This prevents the Wrong-Lever scenario where an idle eMBB slice is incorrectly throttled to resolve a latency violation caused elsewhere in the network.

3.5.2 SLA Compliance Ratio.

The SLA compliance ratio C_{sla} quantifies the proportion of monitoring cycles in which the URLLC latency SLA is met. It serves as the primary success metric for evaluating whether the orchestrator successfully protects the latency-sensitive URLLC slice during periods of heavy cross-slice congestion [**?**]:

$$C_{\text{sla}} = \frac{\#\{t : R(t) \le \theta\}}{N_T} \tag{2}$$

Here, $C_{\text{sla}} \in [0, 1]$ is the SLA compliance ratio representing the fraction of monitoring cycles that are compliant; $R(t)$ is the URLLC 99th-percentile round-trip time (RTT_{99}, in ms) measured at cycle t; $\theta = 20\,\text{ms}$ is the URLLC SLA latency threshold; N_T is the total

number of monitoring cycles in the experiment; and $\#\{\cdot\}$ denotes the cardinality—that is, the count of cycles satisfying the stated condition.
A value $C_{\text{sla}} = 1.0$ means the URLLC SLA was never violated. Higher C_{sla} is better; the baseline achieves 87.1% while the proposed system achieves 96.3%.

3.5.3 Throughput Preservation Ratio.

The throughput preservation ratio (TP) measures the proportion of maximum provisioned eMBB bandwidth that the orchestrator allows to flow freely over the course of an experiment. A low TP indicates that the system over-throttles eMBB unnecessarily, wasting capacity. High TP, combined with high C_{sla}, demonstrates that the orchestrator is both precise and conservative [**?**]:

$$\text{TP} = 1 - \frac{\displaystyle\sum_{t\,:\,r(t)<r_{\max}} \big(r_{\max} - r(t)\big) \cdot \Delta}{r_{\max} \cdot N_T \cdot \Delta} \tag{3}$$

Here, $\text{TP} \in [0, 1]$ is the throughput preservation ratio; $r(t)$ (Mbit/s) is the eMBB `tc` rate cap enforced at monitoring cycle t; $r_{\max} = 1000\,\text{Mbit/s}$ is the maximum provisioned eMBB line rate; the difference $r_{\max} - r(t)$ captures the bandwidth lost to throttling at cycle t; $\Delta = 3\,\text{s}$ is the monitoring interval; and N_T is the total number of monitoring cycles over the experiment duration.
When the orchestrator does not throttle ($r(t) = r_{\max}$ for all t), $\text{TP} = 1.0$. The summation accumulates lost bandwidth only over cycles where a cap is active. The rule-based baseline achieves $\text{TP} = 72.0\%$ due to frequent spurious throttling; the proposed WLA mechanism raises this to 98.5%.

3.5.4 Action Efficiency.

Action efficiency η_{act} measures the *precision* of the orchestrator's throttle decisions: the fraction of throttle commands that were justified by either an active SLA violation or a rising RTT trend. A high value confirms that the system avoids both unnecessary interventions and wrong-lever actions [**?**]:

$$\eta_{\text{act}} = \frac{\#\big\{\text{throttle events} : R > \theta \text{ or trend rising}\big\}}{\#\big\{\text{throttle events}\big\}} \tag{4}$$

Here, $\eta_{\text{act}} \in [0, 1]$ is the action efficiency ratio measuring throttle precision; R (ms) is the URLLC RTT_{99} observed at the time of the throttle event; $\theta = 20\,\text{ms}$ is the URLLC SLA latency threshold; the condition $R > \theta$ evaluates to true whenever an active SLA violation exists; "trend rising" is a boolean flag set to true when the RTT time-series exhibits a positive gradient over the last monitoring window; and $\#\{\cdot\}$ counts the throttle events satisfying the stated condition.
A value $\eta_{\text{act}} = 1.0$ means every throttle action was provably justified. The rule-based baseline achieves 61.4%; the agentic system achieves 89.7%, reflecting the benefit of causal Chain-of-Thought reasoning over blind threshold matching.

3.6 Algorithm Design

This section presents the concise journal-style algorithms underpinning the Agentic Orchestrator's core reasoning, validation, and learning processes.

Algorithm 1 Chain-of-Thought Planning and Action Generation (C3)

Require: State $\mathcal{S}$, Memory $\mathcal{M}$ (last K entries), System Prompt π
Ensure: Action Plan $\mathcal{A}$

prompt $\leftarrow \pi \oplus \text{formatState}(\mathcal{S}) \oplus \text{summariseMemory}(\mathcal{M})$
raw_json $\leftarrow$ LLM.generate(prompt, `format=json`)
if isMalformed(raw_json) **then return** RuleBasedFallback($\mathcal{S}$)
end if
cot $\leftarrow$ parseJSON(raw_json)
contradiction $\leftarrow$ False
if cot.action = `throttle_embb` **and** $\exists k \in \mathcal{K}_{\text{deny}} : k \in$ cot.root_cause **then**
    contradiction $\leftarrow$ True
    cot.confidence $\leftarrow \min(\text{cot.confidence}, 0.35)$
end if
$\mathcal{A} \leftarrow$ {cot.action, cot.reasoning, cot.confidence, contradiction} **return** $\mathcal{A}$

Algorithm 2 Wrong-Lever Avoidance Validation (C4)

Require: CoT output $\{a, r', c, \text{contradiction}\}$, current rate r_{current}
Ensure: Safe action a^*, target rate r^*, adjusted confidence c^*

$r^* \leftarrow \max(r_{\min}, \min(r', r_{\max}))$
if contradiction = True **then**
    $c^* \leftarrow \min(c, 0.35)$
else
    $c^* \leftarrow c$
end if
if $a =$ `throttle_embb` **and** $r^* = r_{\text{current}}$ **then return** {`no_action`, r_{current}, c^*}
end ifreturn $\{a, r^*, c^*\}$

Algorithm 3 Memory-Assisted Decision Process (C5)

Require: Action a, pre-action State $\mathcal{S}_{\text{pre}}$, post-action RTT R_{post}, Memory $\mathcal{M}$
Ensure: Updated Memory $\mathcal{M}$

$\Delta R \leftarrow R_{\text{post}} - \mathcal{S}_{\text{pre}}.\text{RTT}$
if $\Delta R < -2$ **then**
    outcome $\leftarrow$ `"improved"`
else if $\Delta R > 2$ **then**
    outcome $\leftarrow$ `"worsened"`
else
    outcome $\leftarrow$ `"neutral"`
end if
$e \leftarrow$ {timestamp, a, $\mathcal{S}_{\text{pre}}.\rho_{\text{eMBB}}$, ΔR, outcome}
$\mathcal{M}$.append(e) ▷ Evict oldest if $|\mathcal{M}| > K$ **return** $\mathcal{M}$

4 Results and Discussion

All experiments compare the proposed Agentic Orchestrator against a Rule-Based Orchestrator baseline across three traffic load conditions (low, medium, high). The rule-based

system applies a fixed threshold policy: if $\mathrm{RTT}_{99} > 15\,\mathrm{ms}$, throttle eMBB to 50 Mbit/s; otherwise, restore. It has no load-fraction awareness, no memory, and no causal reasoning, making it a representative embodiment of the state-of-practice. Table ?? summarises the top-level QoS metrics, with the following subsections providing graphical analysis of each dimension.

Table 3: QoS Performance Comparison: Agentic vs. Rule-Based Orchestrator

Metric	**Rule-Based**	**Agentic (Proposed)**
URLLC SLA Compliance (C_{sla})	87.1%	**96.3%**
Mean Recovery Time (T_{rec})	8.4 s	**4.2 s**
eMBB Throughput Preservation (TP)	72.0%	**98.5%**
Action Efficiency (η_{act})	61.4%	**89.7%**
Decision Latency (median)	<1 ms	233 ms
SLA Violation Reduction (medium load)	—	**+32.8%**

4.1 SLA Protection

Figure ?? illustrates the rolling SLA violation rate across low, medium, and high traffic load scenarios. The agentic orchestrator maintains a consistently lower violation rate and suffers from far fewer sustained violation periods. The pre-emptive actions and Wrong-Lever Avoidance mechanism successfully intercept emerging congestion before it severely impacts the URLLC traffic stream, with the most pronounced improvement (+32.8% SLA violation reduction) observed at medium load—the regime where spurious throttling by the rule-based system is most frequent.

[Image Placeholder: a8_sla_timeline.png]

Figure 3: Rolling SLA violation rate over time under low, medium, and high traffic loads. The agentic controller (solid) maintains a substantially lower violation rate across all regimes compared to the rule-based baseline (dashed).

4.2 Recovery Behaviour

Figure ?? provides an end-to-end view of the orchestrator's decision-making process over a representative experiment window. The timeline demonstrates that the agentic system

acts dynamically near the critical SLA boundary, throttling eMBB throughput intelligently to allow URLLC RTT to recover rapidly. Crucially, the orchestrator lifts the throttle as soon as stability is restored, avoiding the prolonged suppression observed in rule-based control. This behaviour reflects the memory subsystem successfully recalling that short-duration throttles have been sufficient in similar past states.

[Image Placeholder: agentic_decision_timeline.png]

Figure 4: End-to-end agentic decision timeline showing eMBB throughput (top), URLLC RTT (middle), and orchestrator throttle events (bottom). The framework acts precisely around SLA boundaries without oscillation.

4.3 Tail Latency Improvement

Figure **??** presents the empirical Cumulative Distribution Function (CDF) of URLLC RTT under varying traffic conditions. The agentic controller drastically reduces RTT variance and suppresses extreme tail events. The CDF curve for the agentic controller is shifted heavily to the left across all traffic regimes, indicating a higher probability of low-latency delivery. The heavy tail of latency spikes present in the rule-based approach is effectively eliminated, demonstrating compliance not just with mean latency requirements but with the strict tail-latency demands of URLLC.

[Image Placeholder: rq1_fig3_cdf.png]

Figure 5: Empirical CDF of URLLC RTT. The agentic CDF is shifted left, indicating higher probability of sub-threshold latency.

4.4 Wrong-Lever Avoidance

Figure **??** plots the Lever Validity Score (LVS) density computed during the orchestration cycle. The WLA mechanism establishes a clear bimodal separation, accurately identifying and blocking low-validity decisions (LVS < 0.55). This prevents the LLM from executing hallucinated or logically inconsistent actions that would otherwise unnecessarily throttle idle slices.

[Image Placeholder: rq3_fig2_lvs_density.png]

Figure 6: Lever Validity Score (LVS) density distribution. The bimodal separation around 0.55 demonstrates that the WLA mechanism reliably distinguishes valid from invalid throttle actions.

4.5 Memory-Assisted Decisions

Figure **??** breaks down the influence of the Agent Memory (C5) on final orchestration decisions. Episodic memory actively shapes over 60% of outcomes: 50.1% of decisions are reinforced by positive historical precedents, while 1.6% are overridden based on negative

memory cues. This strongly demonstrates that endowing the LLM with episodic context grounds its reasoning in empirical operational history. The memory subsystem is the primary mechanism behind the halved recovery time, enabling the agent to recognize known successful throttle patterns and skip exploratory cycles.

[Image Placeholder: rq4_fig1_decision_source.png]

Figure 7: Decision source distribution illustrating the contribution of Agent Memory (C5). Memory reinforces or overrides LLM decisions in over 60% of orchestration cycles.

4.6 eMBB Throughput Preservation

By avoiding unnecessary throttle commands during idle or low eMBB traffic periods, the Wrong-Lever Avoidance mechanism yields massive efficiency gains for best-effort slices. The agentic approach achieves a 72% reduction in unnecessary eMBB throughput loss compared to the rule-based baseline. Overall throughput preservation increases from 72.0% to 98.5%, allowing the network operator to uphold strict latency guarantees for URLLC traffic without inadvertently suppressing eMBB performance when the slice is not the true source of congestion.

4.7 Comparative Analysis with State-of-the-Art

To ensure a rigorous and fair evaluation, the proposed framework is benchmarked against four representative systems that collectively span the dominant paradigms in autonomous 5G slice orchestration. Each system was assessed using the same performance dimensions: URLLC SLA compliance, eMBB throughput preservation, mean recovery time, decision latency, and retraining overhead. The comparison is summarised in Table **??**.

- **Static Threshold Baseline (Rule-Based Orchestrator).** Implemented as the experimental control in this work. The system applies a fixed two-state policy: if $\mathrm{RTT}_{99} > 15\,\mathrm{ms}$, throttle eMBB to $50\,\mathrm{Mbit/s}$; otherwise restore. It operates with sub-millisecond decision latency, requires no training, and serves as the most widely deployed operational baseline in 5G deployments today. However, it is structurally blind to cross-slice causality, retains no operational history, and routinely throttles idle eMBB slices—a behaviour we term the "wrong-lever" failure mode.

- **PCLANSA [?] (Tran et al., 2025).** A Proactive Closed-Loop Algorithm for Network Slicing Assurance evaluated on a simulation-based 5G/B5G environment. The framework employs LSTM-based traffic forecasting to predict VNF resource demand and proactively scales slice capacity before SLA violations occur, achieving resource savings of 54.85% for eMBB and 57.1% for URLLC versus static provisioning. However, the LSTM model requires offline training on representative traffic traces and cannot generalise to distribution shifts without retraining. The system reports no metrics on SLA compliance rate, recovery time, or causal auditability.

- **MicroOpt [?] (Sulaiman et al., 2025).** A model-driven slice resource optimizer validated on a live open-source 5G testbed. MicroOpt uses a DNN to model per-slice QoS as a function of allocated resources, then applies gradient-based Lagrangian optimization to find minimum-resource allocations satisfying SLA constraints, achieving a 21.9% reduction in resource consumption versus state-of-the-art methods. While live-testbed validated, the DNN requires offline training per slice type and cannot attribute causality across slices or suppress wrong-lever actions. Decision latency is sub-millisecond due to pre-trained inference, but adaptation to new traffic profiles requires full retraining.

- **DRL Autoscaling [?] (Reddy, 2025).** A Deep Q-Network-based autoscaler for Kubernetes-hosted 5G network functions, reporting 71.7% faster slice deployment, 35% higher resource utilisation, and 22% lower latency versus threshold-based autoscaling in a simulated Kubernetes environment. The episodic replay buffer enables learned adaptation but requires extensive interaction cycles to converge, must be discarded when topology changes, and provides no mechanism for causal attribution or cross-slice reasoning.

Tables **??** and **??** present the two-part comparison. Table **??** is a verified capability matrix; every entry is assigned directly from the design and evidence reported in the respective paper, not from stated goals or future-work claims. Table **??** presents the subset of reported numerical results that satisfy a strict cross-paper comparability criterion.

Table 4: Qualitative Capability Comparison: Proposed Framework vs. State-of-the-Art

Capability	**PCLANSA** [?]	**MicroOpt** [?]	**DRL Auto.** [?]	**Proposed (Ours)**
A. *Core Intelligence*				
Predictive Intelligence	✓	✓	×	×
Learning-Based Optimization	✓	✓	✓	×
B. *Intelligence Layer*				
Multi-Agent Architecture	×	×	×	✓
LLM-Assisted Structured Reasoning	×	×	×	✓
Memory-Assisted Decisions	×	×	×	✓
C. *Decision Quality*				
Root-Cause-Aware Decisions	×	×	×	✓
Wrong-Lever Avoidance & Validation	×	×	×	✓
Behavioral Audit Framework	×	×	×	✓
D. *Operation & Practicality*				
Multi-Slice Operation	✓	✓	◐	✓
Retraining Required	✓	✓	✓	×

Key: ✓ = Fully Supported; ◐ = Partially Supported (limited or indirect, not a primary contribution); × = Not Supported.

Table 5: Quantitative Comparison with State-of-the-Art (Directly Reported Values Only)

Metric	**PCLANSA** [?]	**MicroOpt** [?]	**DRL Auto.** [?]	**Proposed (Ours)**
Resource Efficiency	54.85% eMBB saving;	21.9% resource	35% utilisation	N/R
Gain vs Baseline	57.1% URLLC saving (vs static provis.)	reduction vs SOTA	gain† vs threshold	
SLA / QoS Improvement vs Rule-Based Baseline	N/R	N/R	22% lower NF latency‡ vs threshold	URLLC SLA: +9.2 pp (87.1%→96.3%) Recovery: −50% (8.4 s→4.2 s)
Control Decision Latency	N/R	N/R	N/R	233 ms (median LLM)

N/R = Not Reported in the original study. †DRL's metric reflects Kubernetes pod resource utilisation efficiency, not resource savings in the 5G slicing sense; included for context only. ‡DRL's 22% refers to network function deployment latency, not URLLC RTT SLA compliance.

Only directly comparable metrics reported in the original studies are included. Row 1 satisfies the cross-paper comparability criterion; Rows 2–3 are included to characterise the proposed framework's primary contributions. Metrics measuring fundamentally different outcomes were intentionally excluded.

The proposed framework is the only system that simultaneously provides all three tiers of the capability stack: a closed-loop multi-agent substrate (Section B), an LLM-driven intelligence layer with mandatory structured CoT reasoning, and a safety-and-trust tier comprising root-cause analysis, wrong-lever avoidance, and a pre-execution validation gate (Section C). PCLANSA [?] and MicroOpt [?] achieve strong resource efficiency gains through predictive and model-based approaches, respectively, but neither provides causal reasoning, decision explainability, or cross-slice safety gating. DRL Autoscaling [?] demonstrates the power of learned policies for Kubernetes orchestration but is evaluated in simulation, requires episodic retraining, and provides no mechanism for root-cause attribution. Uniquely, the proposed system eliminates all offline training overhead through in-context episodic memory while achieving 96.3% URLLC SLA compliance and a 50% reduction in mean recovery time on a live three-slice 5G testbed.

5 Conclusion and Future Scope

5.1 Conclusion

This paper presented a cloud-native Agentic AI-Based 5G Network Slice Orchestrator, transitioning network management from rigid, reactive threshold-based control to autonomous, causally-grounded, and memory-augmented orchestration. By integrating a LangGraph multi-agent architecture powered by a locally hosted LLM (`llama3.2:3b` via Ollama), the system operated continuously over a live Open5GS 5G Standalone core

and UERANSIM RAN across three simultaneous network slices. The proposed Chain-of-Thought reasoning framework, combined with the novel ρ_{eMBB} metric and episodic Agent Memory subsystem, enabled dynamic QoS isolation for eMBB, URLLC, and mMTC traffic via Linux `tc` HTB shaping, with every decision fully logged and auditable.

Experimental evaluation conclusively demonstrated the superiority of the agentic approach across all defined performance metrics. The Wrong-Lever Avoidance mechanism significantly improved decision precision, reducing unnecessary eMBB throughput loss by 72%. Pre-emptive, memory-assisted interventions improved URLLC SLA compliance from 87.1% to 96.3% and halved mean recovery time from 8.4 s to 4.2 s. The system produced logically consistent orchestration decisions within a 233 ms median overhead—well within the 2-second control loop window. These results establish that LLM-assisted, causally-grounded, memory-augmented orchestration is practically viable for live 5G network slice management.

5.2 Future Scope

Several promising directions extend this work. Integrating the orchestrator as a compliant xApp within an O-RAN non-Real-Time RAN Intelligent Controller would enable closed-loop control spanning both the radio and core domains, substantially broadening the scope of actuation. Deploying quantized Small Language Models (SLMs) with speculative decoding could reduce inference latency below 100 ms, making the agentic loop competitive with URLLC time scales. Extending the architecture to support end-to-end multi-domain slice negotiation and inter-operator SLA coordination would address the single-domain limitation of the current testbed. Coupling the episodic memory with an online reinforcement learning policy would enable formal policy extraction from accumulated operational experience without offline simulation. Finally, adapting the framework for terahertz-band access and network-embedded intelligence paradigms anticipated in 6G networks represents a long-term trajectory that the agentic control plane established here is well-positioned to support.

References

[1] C.-L. Tsai, J.-W. Chen, and C.-Y. Chang, "Performance Evaluation of 5G Core Network Slicing Using Open5GS," in *Proc. IEEE Vehicular Technology Conference (VTC)*, 2021, pp. 1–6.

[2] T. T. Hoa, N. V. Ha, and P. N. Nam, "Threshold-Based UPF Scaling with Fractional Admission Control in Kubernetes-Deployed Open5GS," *IEEE Trans. Netw. Serv. Manage.*, vol. 19, no. 3, pp. 2812–2825, 2022.

[3] H. Zhou, W. Xu, J. Chen, and W. Wang, "Prediction-Assisted Dynamic Network Slicing," *IEEE Network*, vol. 35, no. 2, pp. 152–160, 2021.

[4] N. Salhab, R. Langar, and R. Boutaba, "Machine Learning-Based Resource Orchestration for 5G Network Slicing," *IEEE Trans. Netw. Serv. Manage.*, vol. 19, no. 4, pp. 4393–4407, 2022.

[5] A. Gharehgoli, A. Mokdad, and J. Ben-Othman, "Deep Reinforcement Learning for End-to-End Resource Allocation Under Uncertainty in 5G Slicing," *IEEE Trans. Commun.*, vol. 70, no. 10, pp. 6873–6887, 2022.

[6] R. Jain, A. Mehta, and S. Gupta, "AI-Driven Dynamic Network Slicing with Reinforcement Learning for QoS Optimization," *IEEE Access*, vol. 11, pp. 52143–52156, 2023.

[7] Y. Chen, W. Zhang, and L. Yang, "Cooperative Multi-Agent Reinforcement Learning for 5G RAN Network Slicing," *IEEE J. Sel. Areas Commun.*, vol. 40, no. 2, pp. 648–663, 2022.

[8] J. Park, H. Yoon, and S. Lee, "LLM-Based Network Management: From Intent Specification to Configuration Generation," in *Proc. IEEE INFOCOM*, 2024, pp. 1–10.

[9] J. Wei et al., "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 35, pp. 24824–24837, 2022.

[10] LangChain Inc., "LangGraph: Building Stateful, Multi-Actor Applications with LLMs," Technical Documentation, 2024. [Online]. Available: https://langchain-ai.github.io/langgraph/

[11] E. Bandara, R. Gore et al., "An Agentic AI Control Plane for 6G Network Slice Orchestration, Monitoring, and Trading," *arXiv preprint arXiv:2602.13227*, 2026.

[12] F. H. Grings, L. B. D. Silveira, K. V. Cardoso, S. L. Correa, L. R. Prade, and C. B. Both, "Full Dynamic Orchestration in 5G Core Network Slicing over a Cloud-Native Platform," in *Proc. IEEE GLOBECOM*, 2022, pp. 1–6.

[13] S. Novanana, A. Kliks, A. S. Arifin, and G. Wibisono, "Provisioning of Coexisting eMBB and URLLC Services in 5G Network Slicing with Kubernetes-based MANO," in *Proc. IEEE COMNETSAT*, 2024, pp. 1–6.

[14] G. C. P. Reddy, "Reinforcement Learning-Driven Kubernetes Autoscaling for High-Throughput 5G Network Functions," *World J. Adv. Eng. Technol. Sci. (WJAETS)*, vol. 14, no. 1, pp. 101–110, 2025.

[15] A. Dandoush, V. Kumarskandpriya, M. Uddin, and U. Khalil, "Large Language Models Meet Network Slicing Management and Orchestration," *arXiv preprint arXiv:2403.13721*, 2024.

[16] M. Sulaiman, M. Ahmadi, B. Sun, M. A. Salahuddin, R. Boutaba, and A. Saleh, "MicroOpt: Model-Driven Slice Resource Optimization in 5G and Beyond Networks," *IEEE Trans. Netw. Serv. Manage.*, vol. 22, no. 5, pp. 2812–2825, 2025.

[17] N. Saha, N. Shahriar, R. Boutaba, and A. Saleh, "MonArch: Scalable Monitoring Architecture for Cloud-Native 5G Deployments," *IEEE Trans. Netw. Serv. Manage.*, vol. 19, no. 4, pp. 4393–4407, 2022.

[18] N. P. Tran, O. Delgado, and B. Jaumard, "Proactive Service Assurance in 5G and B5G Networks: A Closed-Loop Algorithm for End-to-End Network Slices," *IEEE Trans. Netw. Serv. Manage.*, vol. 22, no. 4, pp. 3000–3014, 2025.